E-Commerce Data Integration and Analytics Pipeline Project Documentation and Planning

1. Project Planning and Management

Project Concept and Objectives Project Concept: Building an automated, end-to-end Data ETL Pipeline designed for an e-commerce enterprise. The system ingests disparate product, customer, order, and transaction data from heterogeneous sources including CSV and JSON files, REST APIs, and web sources. It cleanses, validates, and transforms the data before loading it into a relational database structured as a dimensional Star Schema to enable seamless sales reporting and business analytics. Core Objectives:

1. Centralize fragmented data sources into a secure and reliable Data Warehouse.
2. Automate and orchestrate the entire end-to-end data flow using Apache Airflow.
3. Ensure high Data Quality by automatically detecting and handling missing values, duplicates, and format inconsistencies.
4. Design an optimized Star Schema backed by analytical SQL Views to accelerate query performance.

Project Scope In-Scope:

- Multi-source extraction including REST API, web scraping, and local CSV or JSON files.
- Data validation, cleansing, and transformation using Python and Pandas.
- Relational database modeling and Star Schema creation containing Fact Sales, Dim Customer, Dim Product, and Dim Date.
- Workflow automation and scheduling using an Apache Airflow DAG.
- Pipeline containerization with Docker and Docker Compose and continuous delivery via GitHub Actions for CI and CD. Out-of-Scope:
- Building a custom end-user web frontend or dashboard, as the core focus remains on data engineering infrastructure, pipeline reliability, and analytics readiness.

Timeline and Milestones (4-Week Plan)

- Week 1: Design and Data Modeling
    - Requirements analysis and source data profiling.
    - Relational database schema and Star Schema design and ERD.
    - Generating mock sample datasets for testing.

- Week 2: ETL Pipeline Development and Transformation
    - Developing data extraction scripts for REST APIs and local files.
    - Implementing data validation, deduplication, and missing-value handling using Pandas.
    - Writing database loader modules for staging and target tables.
- Week 3: Airflow Orchestration and Quality Assurance
    - Constructing the Airflow DAG following the sequence of Extract, Validate, Transform, Load, and Quality Check.
    - Implementing error handling, logging, and retry mechanisms including retries and retry delay.
    - Creating analytical SQL Views and database indexes for query optimization.
- Week 4: Containerization, CI and CD, and Documentation
    - Packaging the entire pipeline application using Docker and Docker Compose.
    - Setting up the automated build and deployment pipeline using GitHub Actions.
    - Completing comprehensive project documentation, capturing Airflow DAG run execution screenshots, and preparing the final presentation.

Team Roles and Responsibilities (4 Members)

- Ahmed: Data Ingestion Lead responsible for extracting data from REST APIs, web sources, and files, as well as general project coordination.
- Roberto: Data Transformation and Quality Lead responsible for data cleansing, handling missing values and duplicates with Pandas, standardizing data types, and executing data quality checks.
- Mario: Database Architecture and Modeling Lead responsible for relational database modeling, Star Schema design, writing SQL scripts, creating indexes, and building analytical views.
- Haneen: Orchestration and DevOps Lead responsible for building the Airflow DAG workflow, containerizing the application with Docker, and setting up the CI and CD pipeline via GitHub Actions.

Risk Management Matrix

- API Schema or Web Source Drift: Medium probability, high impact. Mitigation strategy is to implement strict schema validation layers that flag and log malformed records without halting the entire pipeline.
- Airflow Task Concurrency and Failures: Medium probability, medium impact. Mitigation strategy is to enable automated retries and retry delay within DAG task parameters, and run local containerized tests using Docker.
- Performance Bottlenecks with Large Datasets: Low probability, high impact. Mitigation strategy is to utilize batched processing in Pandas and apply database indexes on foreign keys and date filters.

Key Performance Indicators (KPIs)

- Data Accuracy and Cleanliness: Achieve a clean data rate of ninety-five percent or higher across all analytical tables with zero unhandled duplicates or missing foreign keys.
- Automation Reliability: Achieve a one hundred percent successful execution rate for scheduled Airflow DAG runs without manual intervention.
- Query Performance: Sub-second retrieval times for sales analytics queries executed against Star Schema SQL Views.

2. Literature Review and Comparative Analysis

Industry Overview In modern data engineering, consolidating heterogeneous data into a unified warehouse is a foundational challenge. Organizations typically choose between off-the-shelf commercial ETL platforms such as Fivetran, Talend, and Apache NiFi, or custom-built programmatic data pipelines.

Solution Comparison

- Commercial ETL Platforms including Fivetran, Talend, and NiFi:
  - Pros: Fast initial setup and drag-and-drop user interfaces.
  - Cons: High licensing costs and limited custom transformation logic capabilities.
- Custom Pipeline Solution including Python, Pandas, Airflow, and Docker:
  - Pros: Complete flexibility and control, open-source with zero licensing cost, and deep integration with custom code.
  - Cons: Requires initial development and testing effort, which is expected and required in graduation projects and professional training.

References and Technical Standards

1. Kimball Dimensional Modeling Methodology: Applied for designing the Star Schema separating fact and dimension tables.
2. Apache Airflow Best Practices Documentation: Followed for constructing modular DAGs, managing task dependencies, and handling retries and logging.
3. Pandas Data Engineering Patterns: Referenced for efficient memory management during batch transformations.
4. Requirements Gathering

Stakeholders

1. Data Analysts: Require clean, structured, and indexed dimensional tables and pre-built SQL views for rapid sales reporting.
2. Business Executives and Decision Makers: Depend on reliable and accurate data warehouse outputs to evaluate business performance and revenue trends.
3. Data Engineers and DevOps Team: Require detailed execution logs, automated alerts, and containerized deployments for easy monitoring and maintenance.

Functional Requirements

- Extraction: Pull product data via REST API or web scraping, and ingest customer and order records from CSV and JSON files.
- Validation and Transformation: Detect and handle missing values, deduplicate redundant records, and standardize data formats and date types.
- Storage and Modeling: Store data in a relational database using a Star Schema containing Fact Sales, Dim Customer, Dim Product, and Dim Date.
- Orchestration: Run an Airflow DAG executing the sequence of Extract, Validate, Transform, Load, and Quality Check.
- Data Quality: Apply quality checks to prevent passing corrupted data into the final tables.

Non-Functional Requirements

- Performance: High query speed achieved by creating SQL indexes on frequently queried columns such as primary keys, foreign keys, and dates.
- Security: Strict separation of database credentials and API keys from codebase using environment variables and Docker secrets.
- Reliability and Maintainability: Comprehensive task logging across all ETL stages, supported by full documentation in the README file and Dockerized reproducibility.